

Feedback Mission: Marsbase Monitoring

Velkommen til din vigtigste opgave som systemingeniør på Mars! Du er ansvarlig for at overvåge kritiske livsdata i en nyoprettet marskoloni. Kolonien har flere autonome sensorenheder placeret rundt om habitatet, som måler:

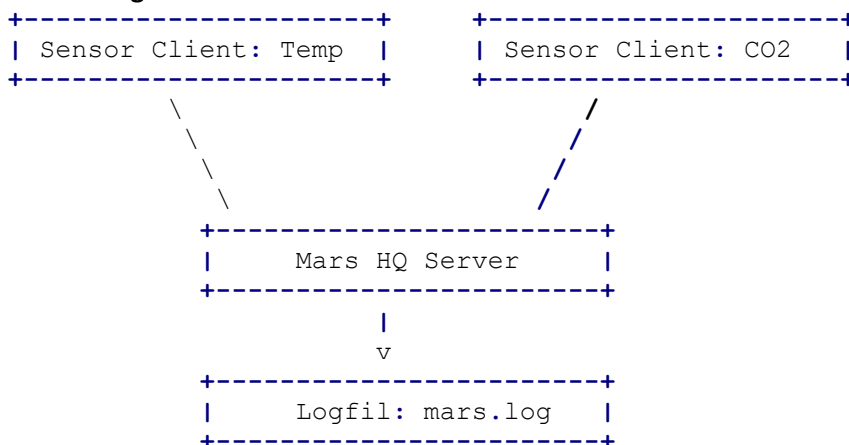
- Temperatur (i grader Celsius)
- Iltindhold (i % af atmosfæren)
- Lufttryk (i hPa)
- CO₂-niveau (i ppm)

Hvis nogen værdier overskrider grænserne, skal HQ (din server) straks reagere og sende en alarm.

Formål med opgaven

- Implementere en multithreaded server, der modtager data fra flere klienter (sensorer)
- Benytte trådpooling til ressourceoptimering
- Reagere på kritiske sensorværdier med alarmer
- Logge alle værdier og events i en logfil
- Bruge fejlhåndtering og sikre korrekt ressourcestyring

Struktur og roller



Scenarie

Sensorene er bittesmå robotter, der er udsendt til forskellige dele af Mars-basen. Hver af dem sender data hvert 5. sekund til HQ. HQ logger alt og sender en "ALARM" hvis noget er farligt for kolonisterne.

Tærskelværdier:

- Temperatur: < -15°C eller > 35°C
- Iltindhold: < 19% eller > 23%
- Tryk: < 800 hPa eller > 1100 hPa
- CO₂: > 2000 ppm

Hvis en værdi krydser grænsen:

ALARM: [SENSOR] value out of range! (value = xxx)

Opgavebeskrivelse (trin for trin)

1. Sensor-klient (Threaded)

- Simuler en sensor ved at sende en ny måling hvert 5. sekund
- Brug fx Random til at generere værdier
- Hver klient sender sin type ("TEMP", "O2" osv.) og en værdi som tekst

Eksempel

```
out.println("TEMP:27.4");  
out.println("CO2:2350");
```

2. Server (med trådpool)

- Brug ExecutorService med fx 5 tråde
- For hver klient: læs linje for linje og parse typen + værdi
- Tjek mod grænser og skriv til logfil
- Hvis alarm: skriv alarm i konsol **og** til klient

3. Logning

- Brug BufferedWriter med FileWriter("mars.log", true)
- Log hver besked med timestamp og sensor-type
[2025-07-18 14:32:01] O2: 22.5
[2025-07-18 14:32:06] CO2: 2100 -> ALARM!

4. Fejlhåndtering

- Brug try-with-resources til alle streams/sockets
- Fang IOException, NumberFormatException
- Udskriv fejl pænt: "[ERROR] Sensor X mistede forbindelsen."

Hjælp og inspiration

- Klient: Brug Socket, PrintWriter, Thread.sleep(5000) og Random
- Server: Brug ServerSocket.accept(), BufferedReader, ExecutorService
- Logger: BufferedWriter eller PrintWriter

Hvis du vil være ekstra sej: Lav en GUI der viser sensorer live (valgfrit)

Afleveringskrav

- Min. 3 sensortyper implementeret (fx TEMP, CO2, O2)
- Trådpool på serveren
- Alarmer og logning virker
- Koden er kommenteret og opdelt i metoder
- README

Arbejde med AI-agent

I denne opgave må AI-agenten gerne implementere mindre, klart afgrænsede dele af løsningen.

I må ikke bede agenten om at udvikle hele Marsbase-systemet på én gang.

Målet er, at I bliver bedre til selv at styre agentens arbejde.

1. Forstå systemet selv

Inden I bruger AI, skal gruppen kunne forklare:

- hvad serverens ansvar er
- hvad sensorklienternes ansvar er
- hvordan flere klienter håndteres samtidig
- hvor ExecutorService skal anvendes
- hvor en sensorforbindelse bør håndteres
- hvilke data klient og server sender til hinanden

Lav eventuelt en lille tegning af systemet.

2. Lav jeres egen overordnede plan

Del opgaven op i mindre dele.

Eksempel:

Server kan modtage én sensor



Flere sensorer kan forbindes



Sensordata kan parses



Grænseværdier kontrolleres



Alarm sendes



Data logges



Fejl og afbrydelser testes

Derefter må I bede AI-agenten om at kontrollere planen.

Eksempel:

Gennemgå vores plan for Marsbase-systemet.

Find manglende trin eller risici.

Du må ikke ændre kode endnu.

I beslutter selv, om agentens forslag skal bruges.

3. Giv agenten en afgrænset opgave

Når agenten skal programmere, skal opgaven være tydelig.

Brug denne struktur:

Opgave:

Hvad skal ændres?

Kontekst:

Hvilke klasser er relevante?

Krav:

Hvad skal løsningen kunne?

Må ikke ændres:

Hvad skal agenten holde sig fra?

Test:

Hvordan ved vi, at ændringen virker?

Eksempel:

Implementér kontrollen af temperaturens grænseværdier.

Arbejd kun med den relevante metode.

Du må ikke ændre serverens trådmodel eller andre sensortyper.

Temperatur under -15 eller over 35 skal give ALARM.

Vis bagefter hvilke linjer du har ændret.

Arbejd med én mindre del ad gangen.

4. Kontrollér agentens ændringer

Efter en ændring skal gruppen:

1. læse diffen
2. forklare hvad der er ændret
3. køre programmet
4. teste ændringen
5. kontrollere at agenten ikke har ændret andre dele unødvendigt

Hvis I ikke kan forklare ændringen, er opgaven ikke færdig.

5. Brug AI som reviewer

Når systemet virker, må agenten gennemgå løsningen.

Eksempel:

Review vores Marsbase-server.

Undersøg især:

- håndtering af flere klienter
- ExecutorService
- sockets og streams
- klienter der afbryder forbindelsen
- logging
- unødvendig kompleksitet

Du må ikke ændre koden.

Giv kun forslag.

I skal selv tage stilling til forslagene.

AI i README

Tilføj et kort afsnit:

AI-agent

Beskriv:

- en opgave I gav agenten
- hvorfor opgaven var afgrænset på den måde
- et forslag eller en ændring fra AI som I accepterede
- et forslag eller en ændring som I ændrede eller afviste
- hvordan I testede at AI-genereret kode virkede

Det skal være kort.

Formålet er ikke at dokumentere alle prompts.

Formålet er at vise, at I kan styre, kontrollere og vurdere en AI-agent.

Bonusudfordringer

- Tilføj en kommandoklient der sender "/status" og får seneste alarmer retur
- Gem alarmhændelser i separat logfil
- Giv sensorer unikke ID'er ("SENSOR-03")

God mission, agent! Mars-kolonien stoler på dig.

